

Universidad Tecnica Estatal de Quevedo
Facultad de Ciencias de la Computacion y Diseno Digital
Carrera de Ingenieria de Software (Redisenio)

Aplicaciones Web – Quinto Nivel
Periodo Academico Presencial 2026-2027 (PPA)

Informe Tecnico

Sistema de Gestion de Inventario
Mercado Municipal de Quevedo

Examen Parcial – Parte Practica

Apellidos y nombres: [COMPLETAR CON TUS APELLIDOS Y NOMBRES]
Cedula: [COMPLETAR CON TU NUMERO DE CEDULA]
Paralelo: [COMPLETAR]
Asignatura: Aplicaciones Web
Fecha del examen: [COMPLETAR CON LA FECHA]
Docente: Dr. Gleiston Ciceron Guerrero Ulloa, Ph.D.

Repositorio del proyecto (URL en una sola linea):

<https://github.com/USUARIO/inventario-mercado-APELLIDO>

Este documento sirve como caratula de identificacion para el criterio de piso P1 y como portada del informe tecnico (criterio P2). **Reemplazar el enlace de arriba por la URL real del repositorio publico antes de compilar la version final.**

Indice

1	Introduccion	2
2	Arquitectura y decisiones de disenio	2
2.1	Arquitectura por capas	2
2.2	Estilo arquitectonico REST	2
3	Modelo de datos	2
3.1	Tabla productos	3
3.2	Tablas usuarios y usuario_roles	3
4	Contrato de la API	3
4.1	Listado paginado – 200 OK	3
4.2	Creacion invalida – 400 Bad Request	4
5	Seguridad: autenticacion y autorizacion con JWT	4
6	Cache-aside con Redis	4
7	Reproducibilidad	5
8	Pruebas realizadas	5
9	Conclusiones	5

1 Introduccion

El Municipio de Quevedo requiere digitalizar el control de productos del mercado central. Este informe documenta el diseno, la implementacion y la verificacion de un servicio web REST construido con Spring Boot 3 sobre Java 21 LTS, que expone el recurso **Producto** bajo un contrato JSON uniforme, valida la entrada en el servidor, aplica borrado logico, integra el patron cache-aside con Redis y protege sus operaciones con autenticacion y autorizacion basadas en JSON Web Token (JWT) [2].

El trabajo cubre los cinco requisitos funcionales exigidos por el enunciado del examen parcial, y se enmarca en los conocimientos de construccion de software y calidad descritos por el SWEBOK Guide [9], como prerrequisito directo del Proyecto Fin de Curso:

- R1. Listado paginado de productos activos, con envoltura de respuesta uniforme.
- R2. Creacion de productos con validacion de entrada en el servidor.
- R3. Eliminacion logica (soft delete) de productos.
- R4. Patron cache-aside con Redis sobre el listado de productos.
- R5. Seguridad basada en JWT con autorizacion por roles.

2 Arquitectura y decisiones de diseno

2.1 Arquitectura por capas

El proyecto sigue una arquitectura por capas clasica, con responsabilidades claramente separadas y una unica direccion de dependencia (controlador → servicio → repositorio), tal como recomiendan Martin [4] y Richards y Ford [7], y siguiendo las convenciones idiomáticas de Spring Boot descritas por Walls [8] (inyeccion de dependencias por constructor, starters de configuracion automatica, DTOs separados de las entidades JPA):

- **Controlador** (`ProductoController`, `AuthController`): expone los endpoints REST, delega toda la logica de negocio al servicio y no contiene acceso a datos.
- **Servicio** (`ProductoServiceImpl`): contiene las reglas de negocio (paginacion, soft delete, cache-aside) y orquesta al repositorio.
- **Repositorio** (`ProductoRepository`, `UsuarioRepository`): unico punto de acceso a la base de datos, mediante Spring Data JPA. Ninguna consulta se concatena manualmente con datos del usuario (se usan siempre parametros nombrados o derivados de metodo), evitando el riesgo de inyeccion SQL descrito en OWASP Top 10:2021 – A03:2021 Injection [6].

2.2 Estilo arquitectonico REST

La API sigue los principios de estilo arquitectonico REST descritos por Fielding [1]: recursos identificados por URI (`/api/v1/productos`), operaciones mapeadas a verbos HTTP (GET, POST, DELETE) y ausencia de estado de sesion en el servidor (autenticacion stateless mediante JWT).

3 Modelo de datos

El esquema se define explicitamente en `db/schema.sql` (tambien incluido en `src/main/resources/schema.sql` para que Spring Boot lo ejecute automaticamente al arrancar). Hibernate se configura en modo **validate**: no genera el esquema, solo valida que las entidades JPA coincidan con las tablas reales.

3.1 Tabla productos

```
CREATE TABLE IF NOT EXISTS productos (  
  id          BIGSERIAL PRIMARY KEY,  
  nombre      VARCHAR(100) NOT NULL,  
  categoria   VARCHAR(50)  NOT NULL,  
  stock       INTEGER      NOT NULL CHECK (stock >= 0),  
  precio      DECIMAL(10, 2) NOT NULL CHECK (precio >= 0.01),  
  activo      BOOLEAN      NOT NULL DEFAULT TRUE,  
  creado_en   TIMESTAMPTZ  NOT NULL DEFAULT now()  
);
```

Las restricciones `CHECK` se aplican a nivel de base de datos como defensa adicional, además de la validación de Bean Validation en el DTO de entrada (sección 4.2).

3.2 Tablas usuarios y usuario_roles

Los usuarios de autenticación también se persisten en PostgreSQL (no están hardcodeados en el código Java), con la contraseña cifrada mediante BCrypt:

```
CREATE TABLE IF NOT EXISTS usuarios (  
  id          BIGSERIAL PRIMARY KEY,  
  username    VARCHAR(50)  NOT NULL UNIQUE,  
  password    VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS usuario_roles (  
  usuario_id BIGINT      NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,  
  rol        VARCHAR(20) NOT NULL,  
  PRIMARY KEY (usuario_id, rol)  
);
```

`db/seed.sql` inserta dos usuarios de prueba con hash BCrypt real (generado con costo 10): `admin` (roles `ROLE_ADMIN` y `ROLE_USER`) y `usuario` (solo `ROLE_USER`).

4 Contrato de la API

Todas las respuestas comparten la misma envoltura JSON, inspirada en el espíritu de uniformidad de *Problem Details for HTTP APIs* [5], aunque adaptada al formato específico exigido por el enunciado:

```
{  
  "success": bool, "data": T, "message": string,  
  "meta": object | ausente, "errors": array | ausente  
}
```

`data` y `message` siempre se serializan (incluso cuando `data` es `null`); `meta` solo aparece en el listado paginado y `errors` solo aparece en las respuestas 400 de validación.

4.1 Listado paginado – 200 OK

```
GET /api/v1/productos?page=0&size=10&sort=nombre,asc  
  
{  
  "success": true,  
  "data": [ { "id": 1, "nombre": "Arroz flor", "categoria": "Granos",  
              "stock": 120, "precio": 0.95, "activo": true,  
              "creadoEn": "2026-07-24T09:15:00-05:00" } ],  
  "message": "Productos obtenidos exitosamente",  
  "meta": { "page": 0, "size": 10, "totalElements": 57, "totalPages": 6 }  
}
```

4.2 Creación inválida – 400 Bad Request

```
{
  "success": false,
  "data": null,
  "message": "Error de validación",
  "errors": [
    { "field": "nombre", "message": "no debe estar vacío" },
    { "field": "precio", "message": "debe ser mayor o igual a 0.01" }
  ]
}
```

La validación se implementa con `@Valid` sobre el DTO de entrada (`ProductoRequestDTO`), anotado con `@NotBlank`, `@Min(0)` y `@DecimalMin("0.01")`. Un `@RestControllerAdvice` (`GlobalExceptionHandler`) captura `MethodArgumentNotValidException` y construye la lista de errores por campo.

5 Seguridad: autenticación y autorización con JWT

La API se protege con JSON Web Token [2]. El flujo es el siguiente:

1. `POST /api/v1/auth/login` autentica usuario/contraseña contra la base de datos (`UserDetailsServiceImpl` + `DaoAuthenticationProvider` + `BCryptPasswordEncoder`) y devuelve un token firmado con HS256, con los roles del usuario como *claim*.
2. `JwtAuthenticationFilter` valida el token en cada petición y reconstruye la autenticación en el `SecurityContext`.
3. `SecurityConfig` exige `ROLE_USER` para el GET y `ROLE_ADMIN` para el POST y el DELETE.

Los casos de error se resuelven con manejadores dedicados, evitando confundir autenticación con autorización (un error frecuente señalado en OWASP Top 10:2021 [6]):

- **401 Unauthorized** (`JwtAuthenticationEntryPoint`): la petición no trae token, o el token es inválido/expirado.
- **403 Forbidden** (`CustomAccessDeniedHandler`): el token es válido, pero el rol del usuario no alcanza para la operación solicitada.

El secreto de firma (`JWT_SECRET`) **no se encuentra embebido en el repositorio**: se lee exclusivamente de una variable de entorno, provista mediante un archivo `.env` (no versionado) a partir de la plantilla `.env.example` con un valor ficticio.

6 Cache-aside con Redis

El listado de productos aplica el patrón cache-aside descrito por Kleppmann [3]:

- `@Cacheable(value = "productos", key = ...)` sobre `ProductoServiceImpl.listar(...)`: en un *cache miss* consulta PostgreSQL, arma la respuesta paginada y la almacena en Redis; en un *cache hit* la sirve directamente desde Redis sin tocar la base de datos.
- `@CacheEvict(value = "productos", allEntries = true)` sobre `crear(...)` y `eliminar(...)`: cualquier escritura invalida todo el cache de listados, para no servir datos obsoletos tras un alta o una baja.
- El TTL del espacio de nombres `productos` se configura en 10 minutos (`application.yml`), como límite superior de obsolescencia si ninguna escritura ocurre.

La clave de cache (`page-size-sort`) evita colisiones entre distintas combinaciones de paginación y orden, ya que cada una es un resultado distinto que debe cachearse por separado.

7 Reproducibilidad

El repositorio se puede clonar y ejecutar en una maquina limpia con un unico comando, sin configuracion manual adicional (mas alla de copiar la plantilla de variables de entorno):

```
git clone https://github.com/USUARIO/inventario-mercado-APELLIDO.git
cd inventario-mercado-APELLIDO
cp .env.example .env
docker compose up -d --build
```

Este comando levanta tres contenedores: **postgres** (con *healthcheck*), **redis** (con *healthcheck*) y **app** (construido desde el **Dockerfile** multi-stage del proyecto, que compila con Maven en la primera etapa y ejecuta con un JRE 21 liviano en la segunda). El servicio **app** espera a que **postgres** y **redis** esten saludables (**condition: service_healthy**) antes de arrancar.

Al iniciar, la aplicacion ejecuta automaticamente **schema.sql** y **data.sql** (identicos a **db/schema.sql** y **db/seed.sql**), dejando la base de datos lista con la tabla **productos** vacia y los dos usuarios de prueba ya sembrados.

8 Pruebas realizadas

La coleccion **docs/requests.http** ejercita los cinco requisitos y los cuatro codigos de estado de error exigidos. Siguiendo el principio de reproducibilidad de la experimentacion en ingenieria de software descrito por Wohlin et al. [10], cada caso de prueba se documenta con su entrada, su codigo HTTP esperado y su criterio de verificacion, de modo que un tercero pueda repetirlo exactamente. La tabla 1 resume los casos cubiertos; las capturas de evidencia correspondientes se encuentran en **docs/capturas/**.

Caso	HTTP	Verificacion
Listado paginado, con token USER	200	meta con totalElements/totalPages correctos
Creacion con datos validos, token ADMIN	201	Header Location presente
Creacion con datos invalidos	400	errors con field/message por cada violacion
Eliminacion de producto existente, ADMIN	200	activo=false en la fila (no se borra)
Eliminacion de id inexistente	404	Mensaje descriptivo
Peticion sin token	401	Mensaje de no autorizado
Peticion con token invalido	401	Mismo manejo que sin token
POST/DELETE con token de rol USER	403	Mensaje de acceso denegado
Misma consulta repetida (paginacion)	–	Segunda vez sin log de <i>cache miss</i>
Alta de producto tras una lectura cacheada	–	Siguiente lectura vuelve a dar <i>cache miss</i>

Tabla 1: Casos de prueba ejecutados con **docs/requests.http**.

9 Conclusiones

El servicio implementado cubre los cinco requisitos funcionales exigidos manteniendo una arquitectura por capas limpia, un contrato de respuesta uniforme y verificable, y dos mecanismos

transversales – cache-aside con Redis y seguridad JWT basada en roles – correctamente aislados en sus propias capas de configuración. El uso de Docker Compose con un único comando de arranque, junto con un esquema de base de datos versionado (`schema.sql/seed.sql`) y una colección de peticiones reproducible (`requests.http`), garantiza que el proyecto pueda verificarse de forma independiente desde una clonación limpia del repositorio, tal como exige el criterio de piso P3.

Referencias

- [1] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [2] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). Technical Report RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [3] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, Sebastopol, CA, 2017.
- [4] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, Boston, MA, 2017.
- [5] Mark Nottingham and Erik Wilde. Problem Details for HTTP APIs. Technical Report RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [6] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks, 2021.
- [7] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, Sebastopol, CA, 2020.
- [8] Craig Walls. *Spring in Action*. Manning Publications, Shelter Island, NY, 6th edition, 2022.
- [9] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.
- [10] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012.